

Submission Worksheet

CLICK TO GRADE

<https://learn.ethereallab.app/assignment/IT114-005-F2024/it114-module-5-project-milestone-1/grade/bna24>

Course: IT114-005-F2024

Assignment: [IT114] Module 5 Project Milestone 1

Student: Bakr A. (bna24)

Submissions:

Submission Selection

1 Submission [submitted] 12/11/2024 1:27:35 PM

Instructions

^ COLLAPSE ^

Overview Video: <https://youtu.be/A2yDMS9TS1o>

1. Create a new branch called Milestone1
2. At the root of your repository create a folder called Project if one doesn't exist yet
 1. You will be updating this folder with new code as you do milestones
 2. You won't be creating separate folders for milestones; milestones are just branches
3. Copy in the code from Sockets Part 5 into the Project folder (just the files)
 2. <https://github.com/MattToegel/IT114/tree/M24-Sockets-Part5>
4. Fix the package references at the top of each file (these are the only edits you should do at this point)
5. Git add/commit the baseline and push it to github
6. Create a pull request from Milestone1 to main (don't complete/merge it yet, just have it in open status)
7. Ensure the sample is working and fill in the below deliverables 1. Note: Don't forget the client commands are /name and /connect
8. Generate the output file once done and add it to your local repository
9. Git add/commit/push all changes
10. Complete the pull request merge from the step in the beginning
11. Locally checkout main
12. git pull origin main

Branch name: Milestone1

Group

100%

Group: Start Up

Tasks: 2

Points: 3

^ COLLAPSE ^

Task

100%

Group: Start Up

Task #1: Start Up

Weight: ~50%

Points: ~1.50

^ COLLAPSE ^

i Details:

Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.



Columns: 1

Sub-Task

100%

Group: Start Up

Task #1: Start Up

Sub Task #1: Show the Server starting via command line and listening for connections

Task Screenshots

Gallery Style: 2 Columns

4

2

1

```
bakra@BakrPC MINGW64 ~/IT114/bna24-IT114-005 (Milestone1)
$ java Project.Server
Server Starting
Listening on port 3000
```

Server Started and Listening on port 3000

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Sub-Task

100%

Group: Start Up

Task #1: Start Up

Sub Task #2: Show the Server Code that listens for connections

Task Screenshots

4

2

1

```

25 // bna24
26 // 10/19/2024
27 private void start(int port) {
28     this.port = port;
29     // server listening
30     System.out.println("listening on port " + this.port);

```

Server code that listens to connection

```

31 try {ServerSocket serverSocket = new ServerSocket(port); }
32 catch(IOException e){} // create the first room
33 while (true) {
34     System.out.println("Waiting for next client");
35     Socket incomingClient = serverSocket.accept(); // blocking action, waits for a client connect
36     System.out.println("Client connected");
37     // wrap socket in a ServerThread, pass a callback to notify the server they're
38     // initialized
39     ServerThread sClient = new ServerThread(incomingClient, this::onClientInitiated);
40     // REUSE the thread (typically an external entity manages the lifecycle and we
41     // don't have the thread REUSE itself)
42     sClient.start();
43     // all is good (scheduling)
44 } catch (IOException e) {
45     System.err.println("Error accepting connection");
46     e.printStackTrace();
47 } finally {
48     shutdown();
49     System.out.println("Closing server socket");
50 }
51 }
52 } // END OF private void start(int port)

```

Rest of the code

Caption(s) (required) ✓Caption Hint: *Describe/highlight what's being shown (ucid/date must be present)***⇒ Task Response Prompt**

Briefly explain the code related to starting up and waiting for connections

Response:

The start(int port) method begins the server and sets the port and creates a default 'LOBBY' room. It then waits in a loop until client connections arrive using a ServerSocket. To handle multiple clients at once, a ServerThread is created when a client connects. The server shut down cleanly if there's an error.

Sub-Task

Group: Start Up

Task #1: Start Up

Sub Task #3: Show the Client starting via command line

100%

🖼 Task Screenshots

4

2

1

```

bakra@BakrPc MINGW64 ~/IT114/bna24-IT114-005 (Milestone1)
$ java Project.Client
Client Created
Client starting
Waiting for input

```

Client Starting via command line

Caption(s) (required) ✓Caption Hint: *Describe/highlight what's being shown***Sub-Task**

Group: Start Up

Task #1: Start Up

Sub Task #4: Show the Client Code that prepares the client and waits for user input

100%

Task Screenshots

Gallery Style: 2 Columns

4

2

1

```
151 // start
152 // 10/10/2024
153 public void start() throws IOException {
154     System.out.println("Client starting");
155
156     // Run CompletableFuture to run listenToInput() in a separate thread
157     CompletableFuture<Void> inputFuture = CompletableFuture.runAsync(this::listenToInput);
158
159     // Wait for inputFuture to complete to ensure proper termination
160     inputFuture.join();
161 } // 10/10/2024 public void start() throws IOException
```

This starts the client

```
152 /**
153  * Listens for keyboard input from the user
154  * 10/10/2024
155  */
156 private void listenToInput() {
157     try (Scanner si = new Scanner(System.in)) {
158         System.out.println("waiting for input"); // moved here to avoid console spam
159         while (isRunning) { // Run until isRunning is false
160             String line = si.nextLine();
161             if (!processClientCommand(line)) {
162                 if (isConnected()) {
163                     sendMessage(line);
164                 } else {
165                     System.out.println(
166                         "Not connected to server (Hint: Type 'connect host:port' without the quotes)");
167                 }
168             } // 10/10/2024 if (!processClientCommand(line))
169             // 10/10/2024 while (isRunning)
170             // catch (Exception ex) {
171             System.out.println("Error in listenToInput()");
172             ex.printStackTrace();
173         }
174         System.out.println("listenToInput thread stopped");
175     } // 10/10/2024 private void listenToInput()
```

Listens for keyboard input

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow leading up to and including waiting for user input

Response:

The start() method starts the client by initializing the client and then starts a separate thread to process user input with a Scanner. It is in continuous loop reading input from the console. Then each line of input is processed to see if it's a particular command, like connecting to a server. The client tries to connect if the command is valid. If the client is connected, the message is sent to the server for regular messages.

End of Task 1

Task



Group: Start Up
Task #2: Connecting
Weight: ~50%
Points: ~1.50

^ COLLAPSE ^

Details:

Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.



Columns: 1

Sub-Task



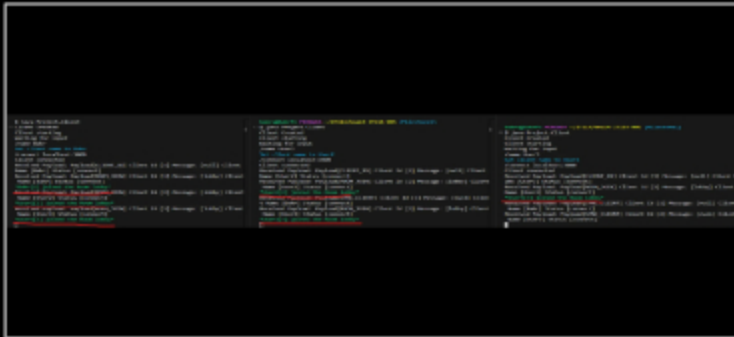
Group: Start Up
Task #2: Connecting
Sub Task #1: Show 3 Clients connecting to the Server

Task Screenshots

Task Screenshots

Gallery Style: 2 Columns

4 2 1



3 Clients: Bakr, User2, User3

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Sub-Task

Group: Start Up

Task #2: Connecting

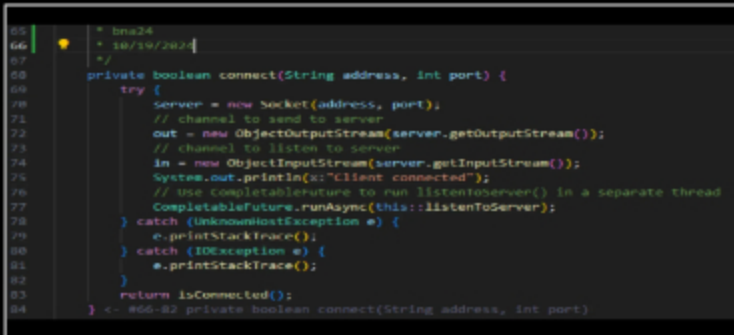
Sub Task #2: Show the code related to Clients connecting to the Server (including the two needed commands)

100%

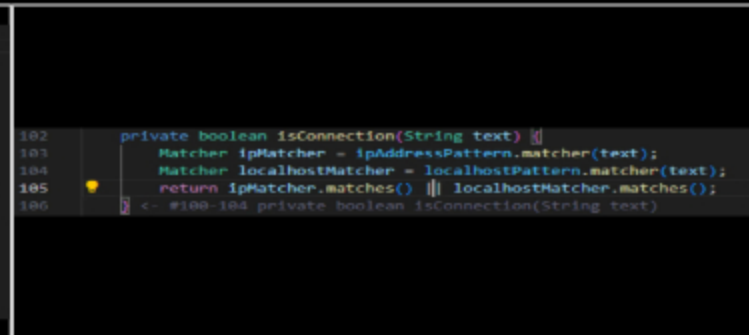
Task Screenshots

Gallery Style: 2 Columns

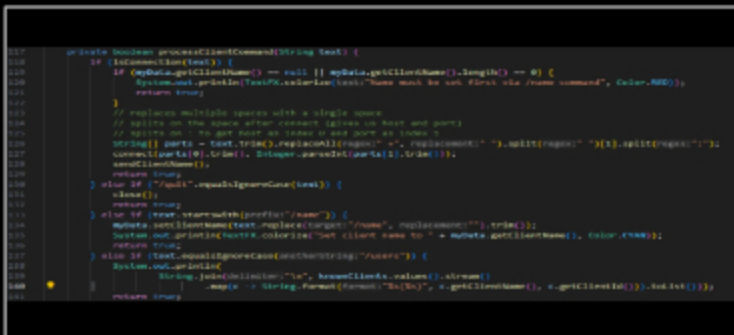
4 2 1



establish a connection



checks if the input text is a valid connection command



processes user input commands

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow

Response:

the connect method of the code creates a Socket and sets up ObjectOutputStream and ObjectInputStream for communication between the client and server. The isConnection(String text) function is just a check to see if the user input is an IP address or "localhost" with a port number. The processClientCommand(String text) method processes user commands, determining whether the input is a valid connection command. Then it checks to see if the client name is valid and if so, it starts the connection and sends the client's name. It also handles /quit for disconnecting and /name for setting the clients' name.

End of Task 2

End of Group: Start Up

Task Status: 2/2

Group



Group: Communication

Tasks: 2

Points: 3

^ COLLAPSE ^

Task



Group: Communication

Task #1: Communication

Weight: ~50%

Points: ~1.50

^ COLLAPSE ^

i Details:

Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.



Columns: 1

Sub-Task



Group: Communication

Task #1: Communication

Sub Task #1: Show each Client sending and receiving messages

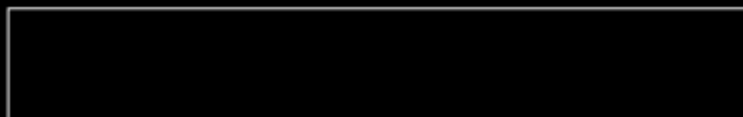
Task Screenshots

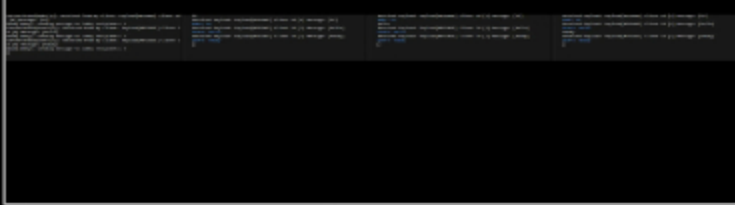
Gallery Style: 2 Columns

4

2

1





Each client sending/receiving a message

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

100%

Group: Communication

Task #1: Communication

Sub Task #2: Show the code related to the Client-side of getting a user message and sending it over the socket

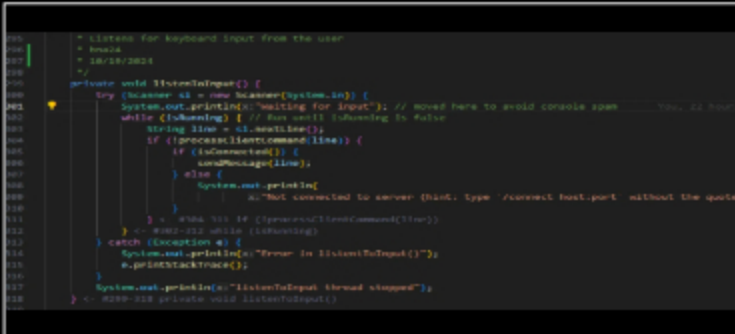
Task Screenshots

Gallery Style: 2 Columns

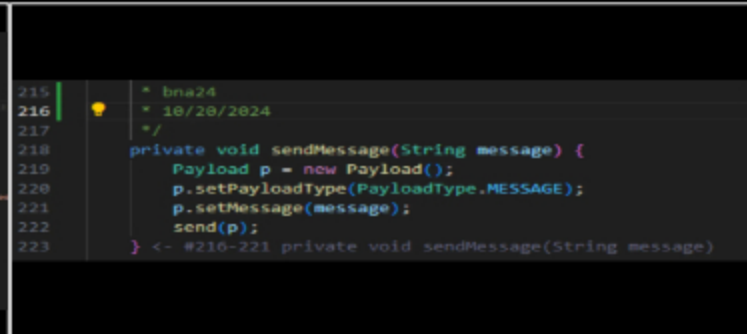
4

2

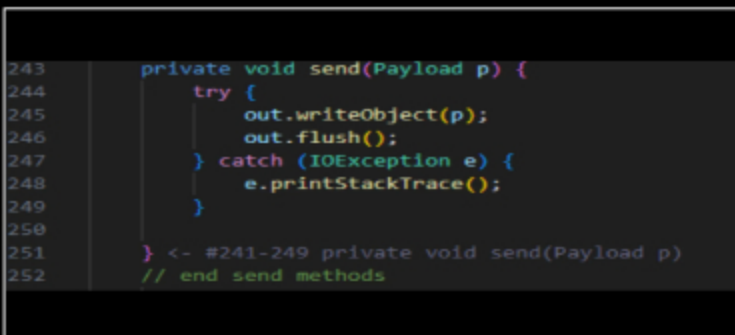
1



Gets the user input



Sends the message



Sends payload over the socket

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown (ucid/date must be present)*

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

The listenToInput() method listens to user input by the client. The method checks if the message is a command when the user types in a message. But if not, it thinks it's a chat message, and calls sendMessage(). With this method, it will create a Payload object with message and then it will invoke the send() method to send it to the server. send() method writes the Payload to the output of the socket in the direction of sending message to server.

Sub-Task

100%

Group: Communication

Task #1: Communication

Sub Task #3: Show the code related to the Server-side receiving the message and relaying it to each connected Client



Task Screenshots

Gallery Style: 2 Columns

4

2

1

```
001 //handle
002 //18/10/2024
003 protected void processPayload(Payload payload) {
004     try {
005         switch (payload.getPayloadType()) {
006             case CLIENT_CONNECT:
007                 ConnectionPayload cp = (ConnectionPayload) payload;
008                 setClientName(cp.getClientName());
009                 break;
010             case MESSAGE:
011                 currentRoom.sendMessage(this, payload.getMessage());
012                 break;
013             case ROOM_CREATE:
014                 currentRoom.handleCreateRoom(this, payload.getMessage());
015                 break;
016             case ROOM_JOIN:
017                 currentRoom.handleJoinRoom(this, payload.getMessage());
018                 break;
019             case DISCONNECT:
020                 currentRoom.disconnect(this);
021                 break;
022             default:
023                 break;
024         }
025     } catch (Exception e) {
026         System.out.println("Could not process Payload: " + payload);
027         e.printStackTrace();
028     }
029 }
```

handle received message from the Client

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

The processPayload method receives the different types of payloads from the clients. It checks the type of message and takes the appropriate action: It sets the client's name when a client connects, relays a message when a message is sent to other clients in the same room, manages those actions when a room is created or joined, and removes a client from the room when they disconnect. And it handles errors.

Sub-Task

100%

Group: Communication

Task #1: Communication

Sub Task #4: Show the code related to the Client receiving messages from the Server-side and presenting them



Task Screenshots

Gallery Style: 2 Columns

4

2

1

```
001 //handle
002 //18/10/2024
003 private void listenToServer() {
004     try {
005         while (isRunning && !isDisconnected()) {
006             Payload fromServer = (Payload) in.readObject(); // blocking read
007             if (fromServer != null) {
008                 // System.out.println(fromServer);
009                 processPayload(fromServer);
010             } else {
011                 System.out.println("Server: disconnected");
012                 break;
013             }
014         }
015     } catch (ClassNotFoundException | ClassNotFoundException cce) {
016         System.err.println("Error reading object as specified type: " + cce.getMessage());
017         cce.printStackTrace();
018     } catch (IOException e) {
019         if (isRunning) {
020             System.out.println("Connection dropped");
021             e.printStackTrace();
022         }
023     } finally {
024         closeServerConnection();
025     }
026     System.out.println("listenToServer thread stopped");
027 }
```

```
001 //handle
002 //18/10/2024
003 private void processMessage(long clientId, String message) {
004     String name = knownClients.containsKey(clientId) ? knownClients.get(clientId).getClientName() : "Room";
005     System.out.println(textFX.colorize(String.format("to: %s", name, message), Color.BLUE));
006 }
```

Listens for messages from the server

formats and displays the messages

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

The listenToServer() method continuously listens for incoming messages. The processMessage() method formats and displays the messages.

End of Task 1

Task

100%

Group: Communication
Task #2: Rooms
Weight: ~50%
Points: ~1.50

^ COLLAPSE ^

Details:

Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.

Columns: 1

Sub-Task

100%

Group: Communication
Task #2: Rooms
Sub Task #1: Show Clients can Create Rooms

Task Screenshots

Gallery Style: 2 Columns

4 2 1



User: Bakr Created a room called House

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Sub-Task

Group: Communication

Task #2: Rooms

Sub Task #2: Show Clients can Join Rooms (leave/join messages should be visible)

100%

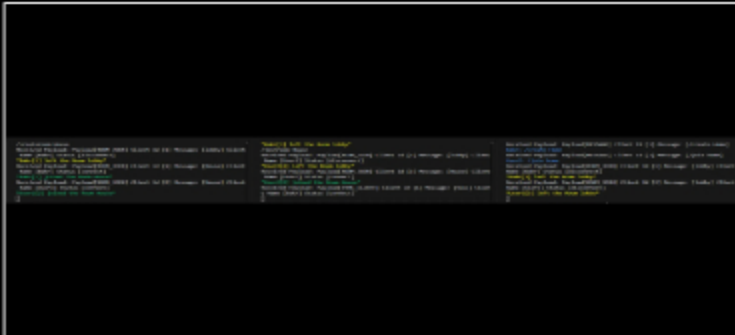
 Task Screenshots

Gallery Style: 2 Columns

4

2

1



Join/leave message

Caption(s) (required) ✓Caption Hint: *Describe/highlight what's being shown***Sub-Task**

Group: Communication

Task #2: Rooms

Sub Task #3: Show the Client code related to the create/join room commands

100%

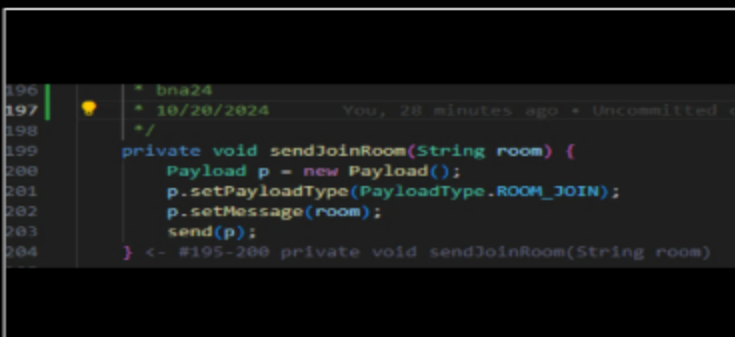
 Task Screenshots

Gallery Style: 2 Columns

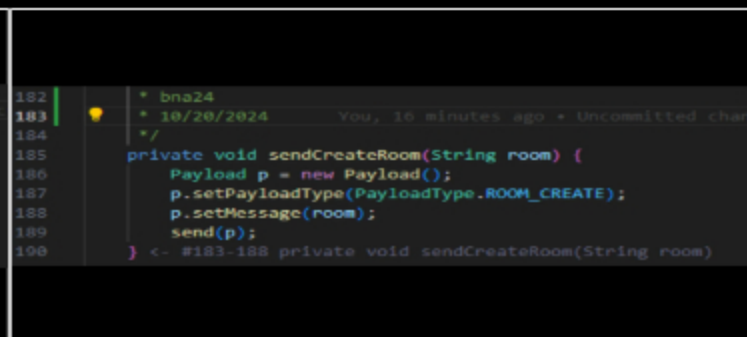
4

2

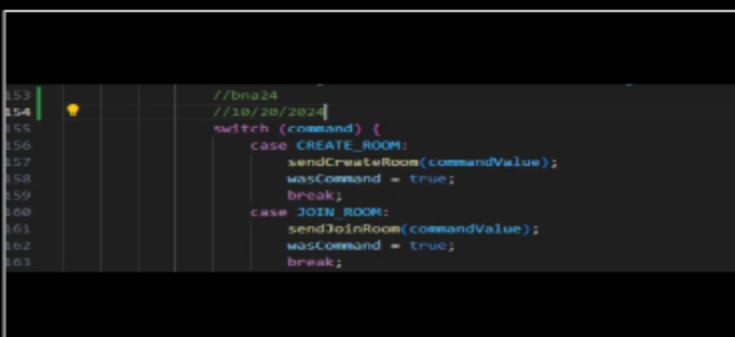
1



sendJoinRoom()



sendCreateRoom()



processClientCommand() that uses create room and join

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

When a user enters a command of type /createroom (name) or /joinroom (name) in the client code, it is processed and the content is fed into the processClientCommand method that parses the command and calls the appropriate handler (sendCreateRoom to generate new room or sendJoinRoom to enter a room already exists). These methods construct a Payload object in which type in the Payload is set either to ROOM_CREATE or ROOM_JOIN and set the room name. Then when the payload is sent to the server, the server is initialized to create or join the room. Requests are processed by the server, and the appropriate messages according to user types are broadcast in the room.

Sub-Task

Group: Communication

Task #2: Rooms

Sub Task #4: Show the ServerThread/Room code handling the create/join process

100%

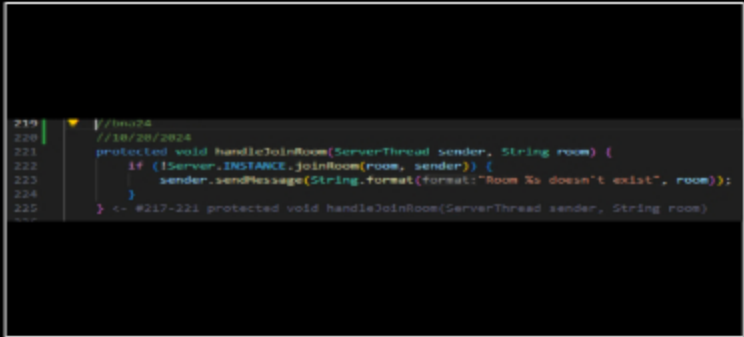
Task Screenshots

Gallery Style: 2 Columns



processPayLoad()

handleCreateRoom()



handleJoinRoom()

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

Response:

The `ServerThread` class processes incoming payloads for the code logic of room creation and joining. When a `ROOM_CREATE` payload is received, it calls `handleCreateRoom`, which checks if the room can be created, if so, the client is added to the room, otherwise an error message is sent. The `handleJoinRoom` method checks if the room exists and adds the client if it does, or sends an error if it doesn't for a `ROOM_JOIN` payload.

Sub-Task

Group: Communication

100%

Task #2: Rooms

Sub Task #5: Show the Server code for handling the create/join process

Task Screenshots

Gallery Style: 2 Columns

4

2

1

```
01 * bma24
02 * 10/20/2024
03 */
04 protected boolean createRoom(String name) {
05     final String nameCheck = name.toLowerCase();
06     if (rooms.containsKey(nameCheck)) {
07         return false;
08     }
09     Room room = new Room(name);
10     rooms.put(nameCheck, room);
11     System.out.println(String.format("Created new Room %s", name));
12     return true;
13 }
14 }
15 }
16 }
17 }
18 }
19 }
20 }
21 }
22 }
23 }
24 }
25 }
26 }
27 }
28 }
29 }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 }
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 }
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
```

createRoom()

```
111 * bma24
112 * 10/20/2024
113 */
114 protected boolean joinRoom(String name, ServerThread client) {
115     final String nameCheck = name.toLowerCase();
116     if (!rooms.containsKey(nameCheck)) {
117         return false;
118     }
119     Room current = client.getCurrentRoom();
120     if (current != null) {
121         current.removedClient(client);
122     }
123     Room next = rooms.get(nameCheck);
124     next.addClient(client);
125     return true;
126 }
127 }
128 }
129 }
130 }
131 }
132 }
133 }
134 }
135 }
136 }
137 }
138 }
139 }
140 }
141 }
142 }
143 }
144 }
145 }
146 }
147 }
148 }
149 }
150 }
151 }
152 }
153 }
154 }
155 }
156 }
157 }
158 }
159 }
160 }
161 }
162 }
163 }
164 }
165 }
166 }
167 }
168 }
169 }
170 }
171 }
172 }
173 }
174 }
175 }
176 }
177 }
178 }
179 }
180 }
181 }
182 }
183 }
184 }
185 }
186 }
187 }
188 }
189 }
190 }
191 }
192 }
193 }
194 }
195 }
196 }
197 }
198 }
199 }
200 }
```

joinRoom()

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

In the `Server` class, the `createRoom` method checks if there is a room with the specified name in the `rooms` map. If it doesn't, it creates a new `Room` object, puts it in the `rooms` map, and logs the creation. The first thing the `joinRoom` method does is verify that the target room exists. If it does, it removes the client from its current room (if any), and retrieves the client's current room. It then adds the client to the specified room, which essentially means the client can switch rooms.

Sub-Task

Group: Communication

100%

Task #2: Rooms

Sub Task #6: Show that Client messages are constrained to the Room (clients in different Rooms can't talk to each other)

Task Screenshots

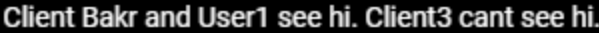
Gallery Style: 2 Columns

4

2

1





Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain why/how it works this way

Response:

The `sendMessage` method in the `Room` class limits client messages to their specific `Room`. It checks the `clientsInRoom` list to see if the client is in the same room and only sends the message to other clients in the same room. What this means is that clients in different rooms cannot see or receive each other's messages. `ConcurrentHashMap` is used to safely manage the clients, so that the messages are delivered correctly without interference from other clients in different rooms.

End of Task 2

End of Group: Communication

Task Status: 2/2

Group

Group: Disconnecting/Termination

Tasks: 1

Points: 3

^ COLLAPSE ^

Task

Group: Disconnecting/Termination

Task #1: Disconnecting

Weight: ~100%

Points: ~3.00

^ COLLAPSE ^

iDetails:

Important: Code screenshots should be fairly concise (try to show only the sections of code relevant to the question)

Capturing all possible code (i.e., including a lot of irrelevant code) can lead to a reduced grade.

↓

Columns: 1

Out Task: ☐ ☐ ☐ ☐ ☐ ☐

Task Screenshots

Gallery Style: 2 Columns

4

2

1

Disconnected without crashing

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Screenshots

Gallery Style: 2 Columns

4

2

1

/quit (not sure if this is considered part of disconnect)

processCommand() disconnect portion

sendDisconnect()

processPayload()

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

The client disconnection process is managed through three main methods: sendDisconnect, processClientCommand, and processPayload. When a user sends a disconnect command, processClientCommand sees that and calls sendDisconnect, which creates a Payload object with type DISCONNECT and sends it to the server to indicate that the user wants to disconnect. This message is then processed by the server and when the client receives a DISCONNECT payload via processPayload, the server matches it to the client's ID and executes the disconnection logic.

Sub-Task

Group: Disconnecting/Termination

Task #1: Disconnecting

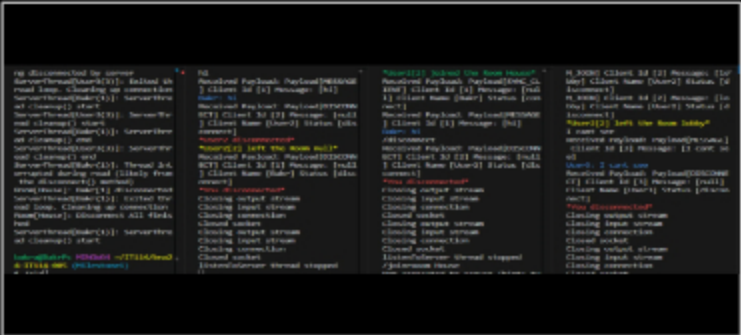
Sub Task #3: Show the Server terminating (Clients should be disconnected but still running)

100%

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Terminating Server

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Sub-Task

Group: Disconnecting/Termination

Task #1: Disconnecting

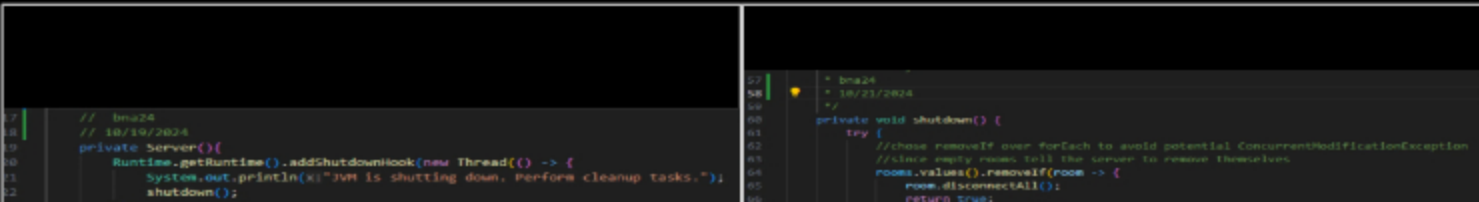
Sub Task #4: Show the Server code related to handling termination

100%

Task Screenshots

Gallery Style: 2 Columns

4 2 1



```
63         }
64     } <- #17-22 private Server
65
66     };
67     } catch (Exception e) {
68         e.printStackTrace();
69     }
70 } <- #18-22 private void shutdown()
71 }
```

Initiates Shutdown

shutdown()

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown (ucid/date must be present)

Task Response Prompt

Briefly explain the code/logic/flow involved

Response:

In the constructor, the server handles termination by using a shutdown hook that calls the shutdown method when the JVM is shutting down. The shutdown method iterates through active rooms and removes each room's clients using removeIf to make sure it is safe to remove without causing errors. This design makes sure that all connections are closed properly and resources are released when the server is terminated.

End of Task 1

End of Group: Disconnecting/Termination

Task Status: 1/1

Group

100%

Group: Misc

Tasks: 3

Points: 1

^ COLLAPSE ^

Task

100%

Group: Misc

Task #1: Add the pull request link for this branch

Weight: ~33%

Points: ~0.33

^ COLLAPSE ^

Task URLs

URL #1

<https://github.com/BakrAbushaar/bna24-IT114-030>

URL

<https://github.com/BakrAbushaar/bna24-IT114-0>

End of Task 1

Task

100%

Group: Misc

Task #2: Talk about any issues or learnings during this assignment

Weight: ~33%

Points ~0.33

^ COLLAPSE ^

Details:

Few related sentences about the Project/sockets topics



≡ Task Response Prompt

Response:

Took me a really long time figuring out where everything is and uploading it to this document. This assignment helped me understand how to filter through long code and figure out what code does what.

End of Task 2

Task



Group: Misc

Task #3: WakaTime Screenshot

Weight: ~33%

Points: ~0.33

^ COLLAPSE ^

Details:

Grab a snippet showing the approximate time involved that clearly shows your repository.

The duration isn't considered for grading, but there should be some time involved.



Task Screenshots

Gallery Style: 2 Columns

4

2

1

Files		Branches	
2 hrs 5 mins	Project/Client.java	2 hrs 45 mins	Milestone1
18 mins	Project/Server.java	5 mins	java-readings-part3-quiz
10 mins	Project/ServerThread.java		
7 mins	Project/Room.java		
5 mins	.gitignore		
8 mins	Project/TestPK.java		
1 min	Project/BaseServerThread.java		
3.6 secs	Project/ClientData.java		
3.5 secs	Project/Payload.java		
3.8 secs	Project/PayloadType.java		
27 secs	Project/ConnectionPayload.java		
2 secs	HS/.gitignore		
0 secs	Project/ServerThread.class		
0 secs	Project/RoomServerThread.class		

WakaTime

End of Task 3

End of Assignment